

16 — Multi-session cross-session manifold alignment

This notebook aligns manifolds across all available sessions and representations. It becomes inferentially meaningful only when at least two, and preferably three or more, sessions have saved embeddings.

Setup and asset index

```
from pathlib import Path
import os
import sys
import yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Edit this path only if your project has moved.
PROJECT_ROOT = Path(
    os.environ.get(
        "LATENT_MANIFOLD_PROJECT_ROOT",
        r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural"
    )
).resolve()

SRC_DIR = PROJECT_ROOT / "src"
if not PROJECT_ROOT.exists():
    raise FileNotFoundError(f"PROJECT_ROOT does not exist: {PROJECT_ROOT}")
if not SRC_DIR.exists():
    raise FileNotFoundError(f"src directory does not exist: {SRC_DIR}")
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
os.chdir(PROJECT_ROOT)

cfg_path = PROJECT_ROOT / "configs" / "publication_upgrade.yaml"
if not cfg_path.exists():
    raise FileNotFoundError(f"Could not find publication config: {cfg_path}")
with cfg_path.open("r", encoding="utf-8") as f:
    pub_cfg = yaml.safe_load(f)
cfg = pub_cfg

# Versioned namespace. This prevents accidental overwriting of older publicat
```

```

PUBLICATION_RUN_LABEL = os.environ.get("PUBLICATION_RUN_LABEL", "publication_u
ALLOW_CANONICAL_PUBLICATION_WRITE = os.environ.get("ALLOW_CANONICAL_PUBLICATION_WRITE", "True")
if PUBLICATION_RUN_LABEL == "publication_upgrade" and not ALLOW_CANONICAL_PUBLICATION_WRITE:
    raise RuntimeError(
        "Refusing to write into the canonical publication_upgrade namespace. "
        "Use a versioned PUBLICATION_RUN_LABEL or set ALLOW_CANONICAL_PUBLICATION_WRITE = True"
    )

class ProjectPaths:
    def __init__(self, root: Path, run_label: str):
        self.root = root
        self.configs_dir = root / "configs"
        self.data_dir = root / "data"
        self.external_dir = root / "data" / "external"
        self.raw_dir = root / "data" / "raw"
        self.interim_dir = root / "data" / "interim"
        self.processed_dir = root / "data" / "processed"
        self.versioned_processed_dir = self.processed_dir / run_label
        self.reports_dir = root / "reports"
        self.tables_dir = root / "reports" / "tables"
        self.figures_dir = root / "reports" / "figures"
        self.html_dir = root / "reports" / "html"
        self.publication_tables_dir = root / "reports" / "tables" / run_label
        self.publication_figures_dir = root / "reports" / "figures" / run_label
        self.manuscript_dir = root / "manuscript" / "top_journal_scaffold"
        for d in [
            self.versioned_processed_dir,
            self.publication_tables_dir,
            self.publication_figures_dir,
            self.manuscript_dir,
        ]:
            d.mkdir(parents=True, exist_ok=True)

paths = ProjectPaths(PROJECT_ROOT, PUBLICATION_RUN_LABEL)

def save_table(df, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
    return path

def save_figure(fig, path, dpi=300):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, dpi=dpi, bbox_inches="tight")
    return path

# Upgrade helper functions shipped with this patch.
from v1_manifold_publication.multisession_core import (
    build_multisession_asset_index,
    candidate_feature_files,

```

```

candidate_feature_files,
choose_best_feature_table,
ready_sessions,
movie_feature_targets,
empirical_null_summary,
safe_read_csv,
safe_table_index,
claim_gate_from_assets,
ensure_table,
)

metadata_path = paths.external_dir / "allen_v1_natural_movie_experiments.csv"
asset_index = build_multisession_asset_index(paths, metadata_path=metadata_path)
save_table(asset_index, paths.publication_tables_dir / "00_multisession_asset_index")

print("Using PROJECT_ROOT:", PROJECT_ROOT)
print("Publication run label:", PUBLICATION_RUN_LABEL)
print("Publication tables:", paths.publication_tables_dir)
print("Publication figures:", paths.publication_figures_dir)
print("Versioned processed dir:", paths.versioned_processed_dir)
display(asset_index)

```

```

Using PROJECT_ROOT: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication run label: publication_upgrade_v3_multisession
Publication tables: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication figures: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Versioned processed dir: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold

```

	session_id	feature_file	feature_shape	
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\allen_v1_natural_movie_experiments.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\allen_v1_natural_movie_experiments.csv
1	500964514	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\allen_v1_natural_movie_experiments.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\allen_v1_natural_movie_experiments.csv
2	501271265	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\allen_v1_natural_movie_experiments.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\allen_v1_natural_movie_experiments.csv

✓ Pairwise cross-session manifold alignment

```

from v1_manifold_publication.manifold_alignment import pairwise_session_align

```

```

alignment_rows = []
status_rows = []

embedding_sessions = ready_sessions(asset_index, "has_embeddings")
print("Sessions with embeddings:", embedding_sessions)

# Analyze each representation separately when available.
all_rep_names = set()
for session_id in embedding_sessions:
    emb_path = Path(asset_index.loc[asset_index["session_id"].astype(str) ==
    try:
        with np.load(emb_path, allow_pickle=False) as emb:
            all_rep_names |= {name for name in emb.files if emb[name].ndim =
    except Exception as exc:
        status_rows.append({"session_id": session_id, "status": "failed", "r

preferred_order = ["cebra", "umap", "isomap", "pca", "pca_full"]
rep_names = [r for r in preferred_order if r in all_rep_names] + sorted(all_
print("Representations considered:", rep_names)

cohort_path = paths.publication_tables_dir / "11_publication_candidate_cohor
metadata = safe_read_csv(cohort_path) if cohort_path.exists() else None

for embedding_name in rep_names:
    embeddings = {}
    for session_id in embedding_sessions:
        emb_path = Path(asset_index.loc[asset_index["session_id"].astype(str)
        try:
            with np.load(emb_path, allow_pickle=False) as emb:
                if embedding_name in emb.files and emb[embedding_name].ndim
                    embeddings[session_id] = emb[embedding_name]
        except Exception as exc:
            status_rows.append({"session_id": session_id, "embedding": embed

    if len(embeddings) < 2:
        status_rows.append({"embedding": embedding_name, "status": "skipped"
        continue

    try:
        alignment = pairwise_session_alignment(embeddings, metadata=metadata)
        if not alignment.empty:
            alignment["embedding"] = embedding_name
            alignment_rows.append(alignment)
    except Exception as exc:
        status_rows.append({"embedding": embedding_name, "status": "failed",

alignment = pd.concat(alignment_rows, ignore_index=True) if alignment_rows e
status = pd.DataFrame(status_rows)

expected_cols = [

```

```

expected_cols = [
    "session_id_a", "session_id_b", "embedding", "alignment_rmse", "mean_cka",
    "median_cka_r", "same_layer", "same_cre_line",
]
if alignment.empty:
    alignment = pd.DataFrame(columns=expected_cols)

save_table(alignment, paths.publication_tables_dir / "16_pairwise_cross_sess
save_table(status, paths.publication_tables_dir / "16_pairwise_cross_session

display(alignment.head(50))
display(status.head(50))

if alignment.empty:
    print("Need at least two sessions with saved embeddings for cross-sessio

```

```

Sessions with embeddings: ['500855614', '500964514', '501271265']
Representations considered: ['cebra', 'umap', 'isomap', 'pca', 'pca_full']

```

	session_a	session_b	n_samples	alignment_rmse	procrustes_scale	n_comp
0	500855614	500964514	900	0.777426	1884.072553	
1	500855614	501271265	900	0.857559	1707.200740	
2	500964514	501271265	900	0.828273	1773.851619	
3	500855614	500964514	900	1.053091	1202.847723	
4	500855614	501271265	900	1.107368	1044.544044	
5	500964514	501271265	900	0.821599	1788.717299	
6	500855614	500964514	900	1.027676	1274.241627	
7	500855614	501271265	900	1.007424	1329.880029	
8	500964514	501271265	900	0.833262	1762.660469	
9	500855614	500964514	900	0.792136	12352.689634	
10	500855614	501271265	900	0.724550	13275.245874	
11	500964514	501271265	900	0.726748	13246.535360	

	embedding	status	reason	n_sessions
0	cebra	skipped	Need at least two sessions with this embedding.	1

✓ Within-layer vs between-layer alignment summary

```

alignment_path = paths.publication_tables_dir / "16_pairwise_cross_session_m

```

```

alignment = safe_read_csv(alignment_path)

if not alignment.empty:
    group_cols = [c for c in ["embedding", "same_layer", "same_cre_line"] if
    metric_cols = [c for c in ["alignment_rmse", "mean_cka_r", "median_cka_r
    agg = {f"mean_{c}": (c, "mean") for c in metric_cols}
    agg.update({f"median_{c}": (c, "median") for c in metric_cols})
    agg["n_pairs"] = (metric_cols[0], "size") if metric_cols else ("embeddin
    alignment_summary = alignment.groupby(group_cols, dropna=False).agg(**ag
else:
    alignment_summary = pd.DataFrame(columns=["embedding", "same_layer", "sa

save_table(alignment_summary, paths.publication_tables_dir / "16_cross_sessi
display(alignment_summary)
if alignment_summary.empty:
    print("Cross-session group summary is empty because fewer than two sessi

```

	embedding	mean_alignment_rmse	median_alignment_rmse	n_pairs
0	isomap	0.994019	1.053091	3
1	pca	0.956121	1.007424	3
2	pca_full	0.747811	0.726748	3
3	umap	0.821086	0.828273	3

✓ Cross-session shared-manifold claim gate

```

n_embedding_sessions = int(asset_index["has_embeddings"].sum()) if not asset
n_full_sessions = int(asset_index["ready_for_full_neural_analysis"].sum()) i
n_pairs = len(alignment) if "alignment" in globals() else 0

claim_gate = pd.DataFrame([
    {
        "claim": "cross-session shared manifold",
        "criterion": "at least 3 sessions with embeddings and nonempty pairw
        "value": f"embedding_sessions={n_embedding_sessions}; pairs={n_pairs
        "status": "supported" if n_embedding_sessions >= 3 and n_pairs >= 3
    },
    {
        "claim": "multi-session neural analysis",
        "criterion": "at least 3 sessions with features, tensors, and embedd
        "value": f"full_neural_sessions={n_full_sessions}",
        "status": "supported" if n_full_sessions >= 3 else "not_ready",
    },
])

```

```
save_table(claim_gate, paths.publication_tables_dir / "16_cross_session_clai
display(claim_gate)
```

	claim	criterion	value	status
0	cross-session shared manifold	at least 3 sessions with embeddings and nonemp...	embedding_sessions=3; pairs=12	supported
1	multi-session	at least 3 sessions with	full neural sessions=2	supported